

Arrays

JavaScript — Fundamentos e Manipulação

PROGRAMAÇÃO FRONT-END

ESTRUTURAS DE DADOS

Arrays

Números, booleanos e strings são os átomos a partir dos quais estruturas de dados são construídas. Muitos tipos de informação requerem mais de um átomo, porém. Objetos nos permitem agrupar valores — incluindo outros objetos — para construir estruturas mais complexas.

Átomos de dados

Números, booleanos e strings formam os tipos primitivos básicos do JavaScript.

Agrupamento

Objetos e arrays permitem combinar múltiplos valores em uma única estrutura.

Complexidade

Estruturas compostas possibilitam representar informações do mundo real com fidelidade.

Definição de arrays

JavaScript fornece um tipo de dado especificamente para armazenar sequências de valores. É chamado de array e é escrito como uma lista de valores entre colchetes, separados por vírgulas.

```
let listOfNumbers = [2, 3, 5, 7, 11];  
console.log(listOfNumbers[2]); // → 5  
console.log(listOfNumbers[0]); // → 2  
console.log(listOfNumbers[2 - 1]); // → 3
```

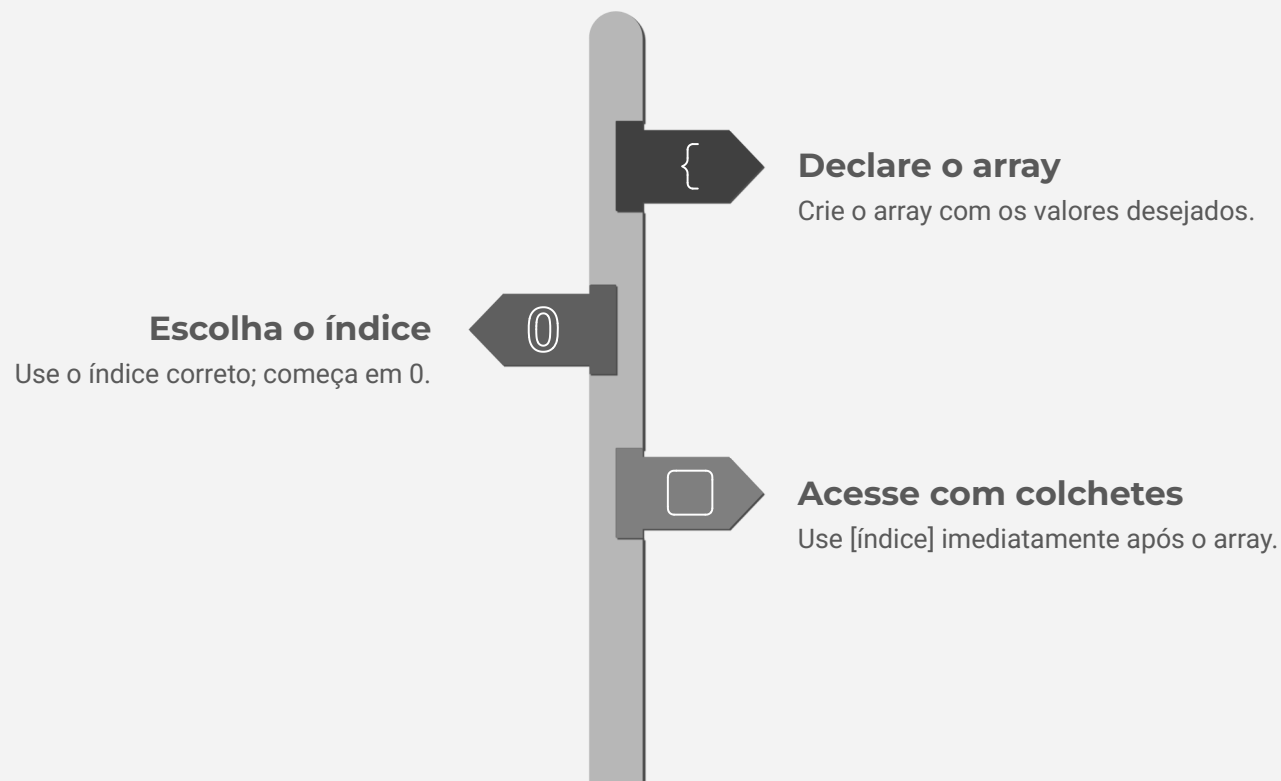
- ❏ Arrays são declarados com colchetes [] e os elementos são separados por vírgulas. Expressões podem ser usadas como índice.

Acesso à elementos de arrays

A notação para acessar os elementos dentro de um array também usa colchetes. Um par de colchetes imediatamente após uma expressão, com outra expressão dentro deles, procurará o elemento na expressão à esquerda que corresponde ao índice dado pela expressão entre colchetes.

Contagem baseada em zero

O primeiro índice de um array é zero, não um, então o primeiro elemento é obtido com `listOfNumbers[0]`. A contagem baseada em zero tem uma longa tradição em tecnologia e de certas formas faz muito sentido, mas leva algum tempo para se acostumar. Pense no índice como o número de itens a pular, contando a partir do início do array.



Propriedades

Quase todos os valores JavaScript têm propriedades. As exceções são `null` e `undefined`. Se você tentar acessar uma propriedade em um desses não-valores, obterá um erro:

```
null.length;  
// → TypeError: null has no properties
```

Valores com propriedades

Strings, números, arrays, objetos e funções — praticamente todos os valores JavaScript possuem propriedades acessíveis.

Exceções: null e undefined

Esses dois não-valores não possuem propriedades. Tentar acessá-las lança um `TypeError` em tempo de execução.

Acessando propriedades

As duas principais formas de acessar propriedades em JavaScript são com um ponto e com colchetes. Sendo `value` uma variável, tanto `value.x` quanto `value["x"]` acessam uma propriedade em `value`.

Notação de ponto

Utilizando a notação de ponto, colocamos o nome da propriedade na sequência — ex.: `value.x`. O nome deve ser um identificador válido.

Notação de colchetes

Utilizando a notação de colchetes, precisamos passar como string (entre aspas) o nome da propriedade — ex.: `value["x"]`. Permite nomes dinâmicos e numéricos.

Propriedades de arrays

Os elementos em um array são armazenados como propriedades do array, usando números como nomes de propriedades.

Notação de colchetes obrigatória

Como não podemos usar a notação de ponto com números, precisamos usar a notação de colchetes para acessar os elementos de um array.

Propriedade length

Arrays têm uma propriedade `length` que nos diz quantos elementos o array tem.

```
let frutas = ["maçã", "banana", "laranja"];
```

```
// Acesso por índice (colchetes)
```

```
console.log(frutas[0]); // → "maçã"
```

```
console.log(frutas[2]); // → "laranja"
```

```
// Propriedade length
```

```
console.log(frutas.length); // → 3
```

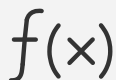
Métodos

Métodos são propriedades que possuem armazenadas em si, funções. Lembrando que JavaScript trata funções como valores, que podem ser armazenadas em variáveis e propriedades.



Propriedade

Um método é, antes de tudo, uma propriedade de um objeto ou valor.



Função armazenada

O valor dessa propriedade é uma função — JavaScript trata funções como cidadãos de primeira classe.



Invocação

Métodos são chamados com parênteses após o nome: `valor.metodo()`.

Métodos de string

Toda string tem uma propriedade `toUpperCase`. Quando chamada, retornará uma cópia da string em que todas as letras foram convertidas para maiúsculas. Existe também `toLowerCase`, que faz o inverso.

Propriedades que contêm funções são geralmente chamadas de métodos do valor a que pertencem, como em "toUpperCase é um método de uma string".

```
let saudacao = "Olá, Mundo!";  
  
console.log(saudacao.toUpperCase()); // → "OLÁ, MUNDO!"  
console.log(saudacao.toLowerCase()); // → "olá, mundo!"
```

`toUpperCase()`

Retorna uma cópia da string com todas as letras em maiúsculas. Não modifica a string original.

`toLowerCase()`

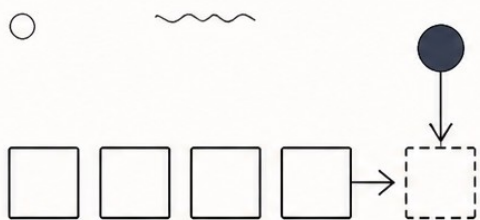
Retorna uma cópia da string com todas as letras em minúsculas. Não modifica a string original.

Métodos de arrays

Os métodos `push`, `pop`, `shift` e `unshift` permitem manipular os elementos das extremidades de um array (início ou fim), funcionando como operações de pilha (stack) ou fila (queue). Todos esses quatro métodos alteram (mutam) o array original no local em que são invocados.

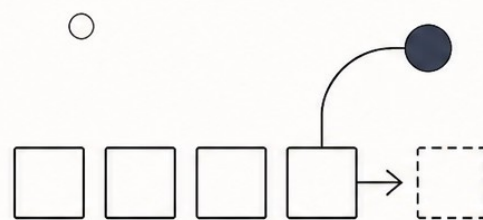
push

(adiciona
ao final)



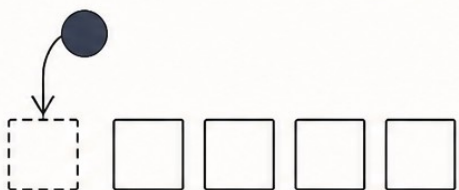
pop

(remove
do final)



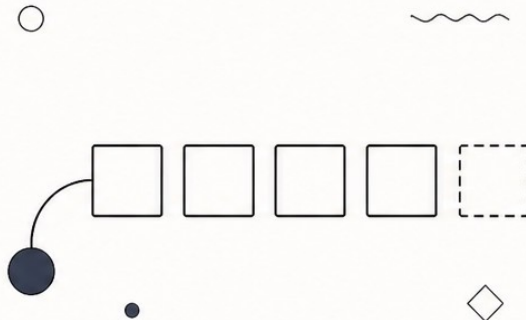
unshift

(adiciona
ao início)



shift

(remove
do início)



push

O método `push()` adiciona um ou mais novos elementos ao final de um array.

Modifica o original

Ele modifica o array original no local.

Retorna o novo comprimento

Ele retorna o novo comprimento (`length`) do array após a inserção.

```
let linguagens = ["HTML", "CSS"];  
let novoTamanho = linguagens.push("JavaScript");  
console.log(linguagens);  
// Saída: ["HTML", "CSS", "JavaScript"]  
console.log(novoTamanho); // Saída: 3
```

```
linguagens.push("TypeScript", "PHP");  
console.log(linguagens);  
// Saída: ["HTML", "CSS", "JavaScript", "TypeScript", "PHP"]
```

pop

O método `pop()` remove o último elemento de um array.

Decrementa o comprimento

Ele decrementa o comprimento do array em 1.

Retorna o elemento removido

Ele retorna o valor do elemento que foi removido.

```
let pilha = ["Primeiro", "Segundo", "Terceiro"];
let elementoRemovido = pilha.pop();
console.log(pilha);
// Saída: ["Primeiro", "Segundo"]
console.log(elementoRemovido);
// Saída: "Terceiro"
```

shift

O método `shift()` remove o primeiro elemento de um array.

Desloca os elementos

Ele desloca todos os elementos subsequentes uma posição abaixo para ocupar o espaço vago no início do array.

Retorna o elemento removido

Ele reduz o comprimento do array em 1 e retorna o valor do elemento removido.

```
let fila = ["Cliente 1", "Cliente 2", "Cliente 3"];  
let atendido = fila.shift();  
console.log(fila);  
// Saída: ["Cliente 2", "Cliente 3"]  
console.log(atendido);  
// Saída: "Cliente 1"
```

unshift

O método `unshift()` adiciona um ou mais elementos ao início de um array.

Desloca os existentes

Ele desloca os elementos existentes para índices superiores para abrir espaço aos novos itens.

Retorna o novo comprimento

Ele retorna o novo comprimento do array após a inserção.

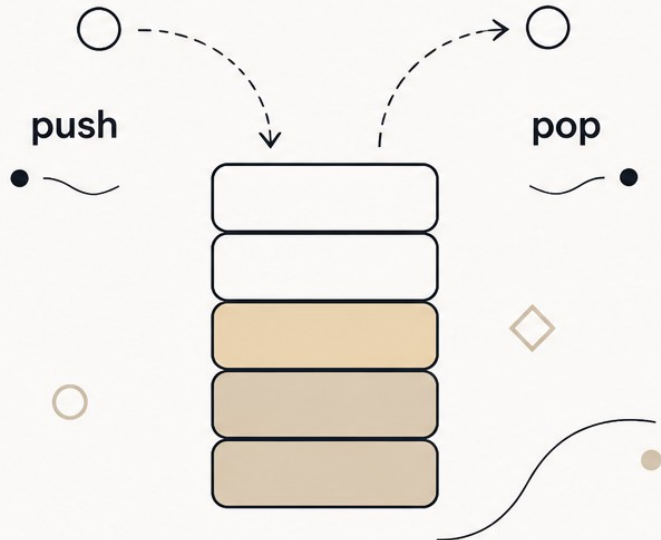
```
let numeros = [3, 4, 5];  
let novoComprimento = numeros.unshift(1, 2);  
console.log(numeros);  
// Saída: [1, 2, 3, 4, 5]  
console.log(novoComprimento);  
// Saída: 5
```

Pilhas e filas

Esses nomes um tanto bobos são os termos tradicionais para operações em uma pilha. Uma pilha, em programação, é uma estrutura de dados que permite empurrar valores para dentro dela e retirá-los na ordem inversa, de modo que o que foi adicionado por último é removido primeiro. Pilhas são comuns em programação.

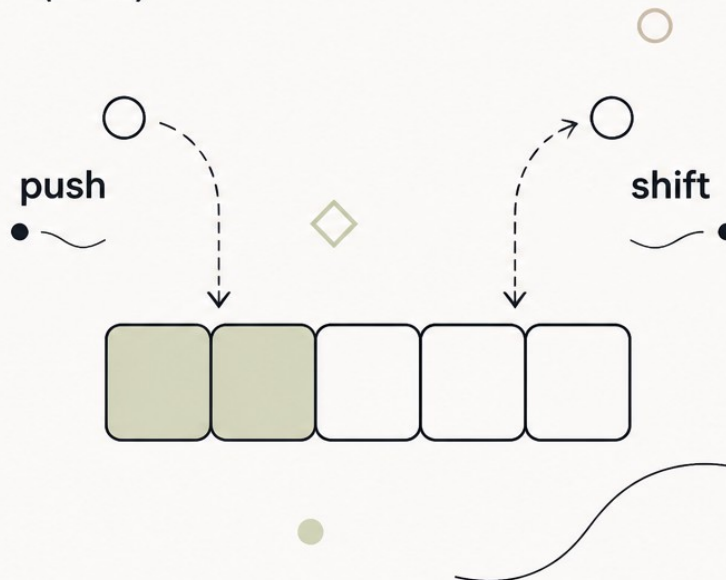
Comparação entre duas estruturas:

Pilha (Stack) usa push e pop —
último a entrar, primeiro a sair
(LIFO);



Comparação entre duas estruturas:

Fila (Queue) usa push e shift —
primeiro a entrar, primeiro a sair
(FIFO)



Loops de arrays

As estruturas de repetição permitem iterar sobre coleções de dados. É fundamental diferenciar o controle por índice numérico (for clássico), a iteração por valores (for...of) e a iteração por chaves/propriedades (for...in).

1

for

Controle por índice numérico. Acesso direto à posição de cada elemento.

2

for...of

Iteração por valores. Sintaxe limpa para percorrer elementos diretamente.

3

for...in

Iteração por chaves/propriedades. Projetado para objetos literais.

For com length

O laço `for` tradicional utiliza uma variável contadora para acessar os elementos através de seus índices numéricos, iterando de 0 até o limite definido pela propriedade `length` (`array.length - 1`).

Principal característica

Oferece controle total sobre o índice e a ordem de iteração (permite saltar elementos, iterar de trás para frente, etc.).

Caso de uso ideal

Quando é necessário manipular o índice diretamente ou modificar os elementos originais do array nas posições específicas.

```
const tecnologias = ["HTML", "CSS", "JavaScript"];

for (let i = 0; i < tecnologias.length; i++) {
  console.log(`Índice ${i}: ${tecnologias[i]}`);
}

// Saída:
// Índice 0: HTML
// Índice 1: CSS
// Índice 2: JavaScript
```

For...of

Introduzido no ES6, o laço `for...of` percorre diretamente os valores (elementos) de estruturas de dados iteráveis (Arrays, Strings, NodeLists do DOM, Maps e Sets).

Principal característica

Sintaxe limpa e legível que elimina a necessidade de gerenciar variáveis contadoras ou acessar a propriedade `length` manualmente.

Caso de uso ideal

Leitura de elementos de um array ou lista de elementos HTML selecionados do DOM quando o número do índice não é relevante.

```
const tecnologias = ["HTML", "CSS", "JavaScript"];

for (let tech of tecnologias) {
  console.log(`Tecnologia: ${tech}`);
}

// Saída:
// Tecnologia: HTML
// Tecnologia: CSS
// Tecnologia: JavaScript
```

For...in

O laço `for...in` foi projetado para percorrer as chaves (nomes de propriedades enumeráveis) de um objeto.

Principal característica

Em arrays, ele itera sobre os índices, mas os retorna como o tipo `String` ("0", "1", "2") e não como `Number`.

Caso de uso ideal

Inspeção de propriedades em objetos literais (`{}`).

```
const curso = {  
  nome: "Técnico em Desenvolvimento de Sistemas",  
  uc: "Programação Front-end",  
  cargahoraria: 120  
};  
  
for (let chave in curso) {  
  console.log(`${chave}: ${curso[chave]}`);  
}  
  
// Saída:  
// nome: Técnico em Desenvolvimento de Sistemas  
// uc: Programação Front-end  
// cargahoraria: 120
```

Mais métodos de arrays

Além dos métodos de manipulação de extremidades, JavaScript oferece outros métodos essenciais para trabalhar com arrays de forma eficiente.

Método	O que faz
<code>slice(início, fim)</code>	Retorna uma cópia superficial de uma porção do array, sem modificar o original
<code>splice(índice, qtd)</code>	Remove, substitui ou insere elementos no array, modificando o original
<code>concat(array2)</code>	Une dois ou mais arrays e retorna um novo array, sem modificar os originais
<code>indexOf(valor)</code>	Retorna o índice da primeira ocorrência do valor, ou -1 se não encontrado
<code>includes(valor)</code>	Retorna <code>true</code> se o array contém o valor, ou <code>false</code> caso contrário
<code>join(separador)</code>	Une todos os elementos do array em uma string, separados pelo separador informado

Considerações finais

Esta foi uma breve introdução à arrays em JavaScript, e por consequência a tópicos que tangenciam nosso próximo assunto, objetos.

Pratique a manipulação de arrays pois é uma habilidade essencial para a construção de programas mais interessantes e eficientes.

Definição

Arrays armazenam sequências de valores entre colchetes, acessíveis por índice numérico baseado em zero.

Métodos essenciais

`push`, `pop`, `shift` e `unshift` manipulam as extremidades do array, mutando o original.

Iteração

`for`, `for...of` e `for...in` oferecem diferentes formas de percorrer coleções de dados.

Próximo passo

Objetos — a estrutura que generaliza e expande os conceitos aprendidos com arrays.